Ders 1 Genişletilmiş Notu: Akış Diyagramları ve Temel Sıralama Algoritmaları

Ders 1 Genişletilmiş Notu: Akış Diyagramları ve Temel Sıralama Algoritmaları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Amacı ve Kapsamı
- 2. Algoritma Nedir, İyi Bir Algoritma Nasıl Olur?
- 3. Zaman ve Yer Verimliliği
- 4. Akış Diyagramı Kuralları
- 5. C ile Program Yazarken Uyulacak İlkeler
- 6. Program = Algoritma + Veri Yapıları
- 7. Kaynaklardan Yararlanmak ve Kendi Çözümünü Tasarlamak
- 8. Bubble Sort
- 9. Exchange Bayraklı Bubble Sort
- 10. for ile while Arasındaki Seçim
- 11. Selection Sort
- 12. Algoritma Analizinde Üç Senaryo
- 13. Doğrusal Arama
- 14. Insertion Sort'a Giriş
- Örnek Sorular

1. Dersin Amacı ve Kapsamı

Veri Yapıları ve Algoritmalar, ikinci sınıfın ikinci döneminde verilen ve sonraki derslerde, özellikle projelerde temel oluşturan bir derstir. Hoca dersin amacını üç başlıkta topladı: temel veri yapılarını öğretmek, algoritma tasarımındaki incelikleri anlatmak ve farklı uygulama alanlarında hangi veri yapısının hangi amaçla kullanıldığını, hangisinin tercih edilmesi gerektiğini gerçek hayattan örneklerle göstermek.

Ders sonunda beklenen kazanımlar şunlardır:

- Doğru ve verimli (efficient) algoritma tasarlayabilmek.
- Kuyruk, yığın, ağaç ve graf yapılarının temel bilgilerini öğrenmiş ve kullanabilecek seviyeye gelmiş olmak.
- Asimptotik analizin ne olduğunu ve iki algoritmayı karşılaştırırken nelere dikkat edileceğini kısaca bilmek. Bunun ayrıntısı bir sonraki dönemin Algoritma Analizi dersinde verilecek.
- Yeni bir probleme yöntem geliştirirken eldeki seçenekleri bilmek ve hangisinin hangi durumda tercih edileceğine karar verebilmek.

Dönem boyunca işlenecek konular sırasıyla şunlardır: algoritma analizi ve temel notasyonlar, özyinelemeli (recursive) algoritmalar, listeler ve bağlı listeler (linked list), kuyruk (queue) ve yığın (stack), ağaçlar, heap ağaçları ve heap sıralama algoritması (heap sort), öncelikli kuyruk (priority queue), graflar, metin (string) arama algoritmaları ve sıralama algoritmalarının

birbirleriyle karşılaştırılması. Hash veri yapısı bu dönem işlenmeyecek, Algoritma Analizi dersinde anlatılacak.

Dersin anlatımında akış diyagramı sık kullanılacak. Birçok algoritmik problem akış diyagramıyla çözülecek. Sözde kod (pseudo code) ve C programlama dili de algoritmaları anlatırken en çok başvurulacak gösterimler olacak. Hoca kaynak olarak *Introduction to Algorithms* kitabını önerdi. Ek kaynaklar olarak Levitin'in *Introduction to the Design and Analysis of Algorithms*, Skiena'nın *The Algorithm Design Manual* ve Sedgewick'in *An Introduction to the Analysis of Algorithms* kitaplarını saydı.

> **Ders sonrası not:** Hoca *Introduction to Algorithms* kitabını "MIT tarafından yayınlanmış" diye tanıttı. Kitap büyük olasılıkla Cormen, Leiserson, Rivest ve Stein'ın MIT Press'ten çıkan kitabıdır (CLRS). Hoca Sedgewick'in kitabının Java ağırlıklı olduğunu söyledi. Java ağırlıklı olan büyük olasılıkla Sedgewick ve Wayne'in *Algorithms* kitabıdır; *An Introduction to the Analysis of Algorithms* (Sedgewick ve Flajolet) büyük olasılıkla matematiksel analize odaklanır.

2. Algoritma Nedir, İyi Bir Algoritma Nasıl Olur?

Bilgisayar bilimlerinde ve teknolojik problemlerde bir problemi çözmek için izlediğimiz yola algoritma diyoruz. Bizim için değerli olan çözüm, verilen bir girdi için istenen çıktıyı üreten ve bu sonucu sonlu zamanda üreten çözümdür.

Bir algoritma tasarlarken dikkat edilecek hususlar şunlardır:

- **Doğruluk (correctness):** Algoritma verilen girdi için istenen çıktıyı, var olan tüm senaryolarda üretebilmelidir. Birinci öncelik budur. Algoritma önce doğru çalışmalıdır.
- **Kolay gerçeklenme ve anlaşılabilirlik:** Çok karmaşık bir adım dizisi uygularsanız başka birinin onu anlaması kolay olmaz.
- **Değiştirilebilirlik:** Yeri geldiğinde algoritmaya kolayca müdahale edilebilmelidir. Çözümünüzü başka birine devrettiğinizde, o kişinin işi kaldığı yerden sürdürebilmesi için kodunuzu ve algoritmanızı anlayabilmesi gerekir. Hoca bunun önemini grup çalışmalarında daha iyi anlayacağımızı söyledi.
- **Probleme uygun programlama dili:** Tasarlanan algoritma her dille kodlanabilir. Ama problemin türüne göre bazı diller avantaj sağlar. Gömülü sistemlerde C, veriyle uğraşırken kütüphaneleri zengin olan Python, olay güdümlü (event driven) programlamada Java ya da C++ gibi diller tercih edilebilir.

  > **Hocanın vurgusu:** Bir problemi çözerken birinci önceliğimiz kodlamasını yapmak değil, algoritmasını tasarlamaktır.

Bir algoritmayı ifade etmenin yolları şunlardır:

- Akış diyagramı (flowchart).
- Sözde kod (pseudo code).
- Programlama dili.
- Sözle tarif. Bu, birine genel olarak bir şey anlatırken kullandığımız yöntemdir.

İlk üçü temel gösterimlerdir. Derste ağırlıklı olarak akış diyagramlarıyla ilerlenecek, yeri geldiğinde sözde kodun avantajlarından da yararlanılacak.

3. Zaman ve Yer Verimliliği

Verimlilik açısından iki önemli ölçüt vardır:

- **Zaman verimliliği (time efficiency):** Algoritmanın ne kadar hızlı çalıştığı, işini ne kadar kısa sürede bitirdiği.
- **Yer verimliliği (space efficiency):** Algoritmanın işini yaparken ne kadar kaynak, özellikle ne kadar bellek alanı kullandığı.

Birinci öncelik zaman karmaşıklığıdır (time complexity). Algoritma doğru çalışsa bile çok uzun sürede sonlanıyorsa verimli olmadığı için seçenek olmaktan çıkabilir. İki algoritmayı karşılaştırırken ikisi de doğru çalışıyorsa zaman karmaşıklığı düşük olanı seçeriz. Aynı hızda çalışı-

yorlarsa bellek tüketimine bakar, tüketimi düşük olanı seçeriz. Bellek kısıtımız varsa bellek verimliliğini öne alabilir, zaman karmaşıklığını ikinci plana atabiliriz.

4. Akış Diyagramı Kuralları

Hoca derste kullanılacak akış diyagramı sembollerini tek tek hatırlattı.

4.1 Başlangıç ve bitiş

Bir algoritma bir noktada başlar ve bir noktada biter. Her akış diyagramında bir başlangıç (start) ve bir bitiş (stop) noktası vardır. Bir akış diyagramının birden fazla başlangıç ya da bitiş noktası olmaz.

> **Hocanın vurgusu:** Değerlendirmelerde en çok sorun yaşanan kısımlardan biri doğru akış diyagramı sembollerini kullanmaktır. Bu konuya şimdiden özen gösterin.

4.2 İşlem kutusu

Dört işlem, atama gibi işlemler işlem kutusuna yazılır. Bir kutuya birden fazla işlem yazılabilir.

4.3 Karar kutusu (if)

Karar kutusuna kontrol ifadesi yazılır: büyüktür, küçüktür, büyük eşittir, küçük eşittir, eşittir, eşit değildir. Kutudan iki kol çıkar. Biri koşul doğruysa (true) ne yapılacağını, diğeri koşul yanlışsa (false) ne yapılacağını gösterir.

En çok yapılan hata, true kolunu boş bırakıp false koluna iş yazmaktır. Bir `if`'in true kolu boşken false kolu dolu olmaz. True kolu mutlaka dolu olmalıdır, gerekirse false kolu da doldurulur. Ters bir durum varsa koşul ters çevrilir. Örneğin yalnız `x > y` yanlışken bir iş yapılacaksa koşul `x <= y` diye yazılıp iş true koluna konur.

> **Hocanın vurgusu:** Bir `if` kullanıyorsanız true kolu mutlaka dolu olmalıdır. Hangi kolun true, hangisinin false olduğunu da her zaman yazın.

4.4 for döngüsü

Adım sayısı belli olan tekrarlı işlerde `for` döngüsü kullanılır. Sembolün içine bir döngü değişkeni (örneğin i), başlangıç değeri, bitiş değeri ve her adımdaki artış ya da azalış miktarı yazılır. Döngüye önceki akıştan bir noktadan girilir, her adımda yapılacak işler döngü gövdesine yazılır, döngü bitince başka bir noktadan çıkılıp akışa devam edilir. Geriye doğru sayarken, örneğin 100'den 5'e inerken adım -1 yazılır.

4.5 while döngüsü

Adım sayısı belli olmayan işlerde `while` döngüsü kullanılır. `if`'ten farklı olarak, döngü kolu karar sembolünün orta noktasından çıkar ve üst kenara geri bağlanır. `while`'ın içine adım sayısından çok, sağlandığı sürece devam edilecek bir ya da birden fazla koşul yazılır. Örneğin `x < y` yazarsanız x, y'den küçük olduğu sürece; `x == y` yazarsanız x ile y eşit olduğu sürece döngüde kalırsınız. Döngüye yukarıdan girilir, iş bitince aşağıdan çıkılır.

`while` döngüsünde önce kontrol, sonra iş yapılır. Diğer biçimde (`do-while` benzeri yapı) önce mutlaka iş yapılır, sonra koşul kontrol edilir; koşul sağlanıyorsa yine işe dönülür.

Soru: C'de `for`'un bitiş kısmına farklı ifadeler yazabiliyoruz. Akış diyagramında bitiş kısmına yalnız bitiş noktası mı yazılır, yoksa bir koşul da yazılabilir mi?

Cevap: Genellikle buraya başlangıç ve bitiş noktaları yazılır. Programlama dilinde de mümkün olduğunca böyle yazılması istenir. Örneğin bitiş noktanız $x \cdot y$ ise buraya $x \cdot y$ yazılmaz; önce `n = x * y` hesaplanır, buraya `n` yazılır. "$x \leq y$ olduğu sürece" gibi bir koşul yazmak tercih edilen bir şey değildir. `for` adım sayısı belli olan bir iş için kullanıldığından başlangıç 1 olur ve n kaçsa oraya kadar gidilir; ya da başlangıç 100, bitiş 5 olur ve adım -1 ile geriye doğru gelinir.

Yani bu alanlara genellikle bir değişken ya da sabit bir sayı yazılır. Koşul yazmak gerekiyorsa mutlaka `while` döngüsü kullanılmalıdır.

Soru: `if` kontrolünde true ya da false kolunun hangi yönde olduğu fark eder mi?

Cevap: Hiç fark etmez. Genelde true sağa, false sola yazılabiliyor ama bunun bir önemi yok. Önemli olan hangi kolun true, hangisinin false olduğunu her durumda belirtmektir.

5. C ile Program Yazarken Uyulacak İlkeler

Ders boyunca programlama dili olarak C kullanılacak. Hoca iyi bir C programının şu ilkelere uymasını istedi:

- ANSI C standartlarına uygun yazın. Böylece program hangi platformda derlenirse derlensin hatasız çalışır.
- Global değişkenlerden uzak durun. Değerleri fonksiyonlara parametre olarak aktarın.
- Değişken adlarını anlamlı seçin. `a`, `b`, `c`, `aa`, `bb` gibi taşıdığı bilgiyi anlatmayan adlardan kaçının. Fonksiyon adları da fonksiyonun yaptığı işi anlatmalıdır.
- Modüler programlama yapın. Problemi alt problemlere ayırın ve her alt problem için fonksiyon yazın. Böylece kod yeniden kullanılabilir (reusable) olur.
- Mümkün olduğunca dinamik bellek ayırın (dynamic memory allocation). Statik yer ayırmaktan kaçının, gerektiği kadar yer ayırın.
- Gerçekten açıklama gereken yerlere açıklama satırı (comment) koyun.

Bu ilkeler hem iş hayatında hem de kodunuza daha sonra baktığınızda ya da başkası kodunuzu kullanmak istediğinde işinizi kolaylaştırır.

6. Program = Algoritma + Veri Yapıları

Niklaus Wirth'e göre bir program iki temel bileşenden oluşur: bir algoritma ve bu algoritmayı gerçekleştirmek için kullanılan veri yapıları. Bu dersin en önemli kazanımı veri yapılarını iyi tanımak ve doğru kullanabilmektir. Çünkü veri yapılarını doğru kullanmak hem zaman hem de yer verimliliği sağlar.

> **Ek bilgi (derste anlatılmadı):** Bu görüş Wirth'in 1976'da yayımladığı *Algorithms + Data Structures = Programs* kitabının adıyla büyük olasılıkla özdeşleşmiştir.

Temel veri yapıları şunlardır:

- **Dizi (array):** Aynı tipteki değerleri saklayan temel yapı. Matrisler için iki boyutlu (two-dimensional), küp için üç boyutlu dizi kullanılır. C, 4 ya da 5 boyutlu dizilere de izin verir. Veriler bellekte arka arkaya saklanır ve her elemana doğrudan erişilir. Ama dizinin bazı dezavantajları vardır. Bazı durumlarda dizi yetmez ve dizinin üzerine kurulan ya da diziye alternatif olan başka yapılar kullanılır.
- **Bağlı liste (linked list).**
- **Yığın (stack):** Temeli dizi olan, ama kullanım biçimi farklı olan bir yapı. Bağlı listeyle de gerçeklenebilir. Son giren ilk çıkar (last in first out, LIFO).
- **Kuyruk (queue):** Yığın gibi dizi ya da bağlı listeyle gerçeklenebilir. İlk giren ilk çıkar (first in first out, FIFO).
- **Graf (graph) ve ağaç (tree):** Ağaç, grafın özel bir hâlidir. Bu dönem sık karşılaşılacak yapıların başında gelirler.
- **Hash:** Bu dönem işlenmeyecek.

Birçok iş sıralanmış veri üzerinde çok daha verimli yapılır. Şimdiye kadar bubble sort ve selection sort görüldü, quick sort ve merge sort duyulmuş olabilir, insertion sort'u bazıları gördü. Dönem içinde bunlara ek olarak counting sort, heap sort, shell sort gibi algoritmalar; hangi durumda tercih edildikleri, artı ve eksileri, karmaşıklık açısından birbirlerine üstünlükleri tartışılacak.

Temel algoritmaların başında arama algoritmaları gelir. Doğrusal arama (linear search, diğer adıyla sequential search) ve ikili arama (binary search) görüldü. Hash mekanizmasıyla da arama yapılabildiği ileride öğrenilecek. Sıralama ve aramanın yanında ekleme, silme ve güncelleme gibi dizilerde kullanılan temel işlemler, diğer veri yapıları için de bu derste öğrenilecek.

7. Kaynaklardan Yararlanmak ve Kendi Çözümünü Tasarlamak

İkinci sınıfın ikinci dönemindeyiz. Artık yalnız doğru değil, mümkün olduğunca verimli çalışan algoritmalar bekleniyor: gereksiz `if`'lerden ve gereksiz döngülerden uzak, zaman karmaşıklığı mümkünse daha düşük çözümler.

Kaynak okumak ve kaynaklardan yararlanmak desteklenir. Ama kaynaktaki çözümü birebir uygulamak olumlu değerlendirilmez. Bu problemlerin hepsinin internette bir çözümü vardır. Önemli olan sizin çözümü kendiniz tasarlayabilmenizdir. Böylece işi daha iyi anlarsınız. İş hayatında her şeyi kendiniz tasarlamayacak, hazır çözümler kullanacaksınız. Ama kendiniz tasarlamayı öğrenirseniz bir kaynaktaki hatayı fark edebilirsiniz. Çünkü her kaynaktaki çözüm doğru olmayabilir. Bir kaynaktan alıntı yapan öğrenciler, istenen iş için uygun olmayan ya da düşünmedikleri bir hatayla karşılaşabiliyor.

8. Bubble Sort

Sıralama en önemli problemlerden biridir. Veriyi genelde sıralı kullanmak isteriz, çünkü sıralı veri arama dahil birçok işi kolaylaştırır. Bubble sort, birinci sınıfta görülen bir sıralama algoritmasıdır. Yan yana gelen elemanların büyüklüklerine göre yer değiştirmesi ilkesine dayanır.

8.1 Örnek: 3, 5, 1, 2, 7, 4

Hoca birinci turun ilk adımlarını anlattı: 3 ile 5 karşılaştırılır, 5 daha küçük olmadığı için yerinde kalır. 5 ile 1 karşılaştırılır, 1 daha küçük olduğu için yer değiştirir. Sonra 5 ile 2 karşılaştırılır, yine yer değişir. Birinci tur böyle tamamlanır. Birinci turun tamamı aşağıdadır (ara adım eklendi):

Karşılaştırma	Sonuç	Dizi
3 ile 5	yerinde	3, 5, 1, 2, 7, 4
5 ile 1	yer değişir	3, 1, 5, 2, 7, 4
5 ile 2	yer değişir	3, 1, 2, 5, 7, 4
5 ile 7	yerinde	3, 1, 2, 5, 7, 4
7 ile 4	yer değişir	3, 1, 2, 5, 4, 7

Birinci turun sonunda en büyük eleman (7) en sona itilmiştir. Ama tüm dizi sıralanmamıştır. Bu yüzden iç içe iki döngü gerekir: her tur, kalan elemanlar içindeki en büyük elemanı uca taşır. İkinci tur ikinci en büyük elemanı, üçüncü tur üçüncü en büyük elemanı yerine koyar.

Küçük elemanlar da her turda birer adım başa yaklaşır. Birinci turun sonunda 1, 5'in eski yerini almıştı. İkinci turda 3'ün yerini alır. İkinci tur (ara adım eklendi):

Karşılaştırma	Sonuç	Dizi
3 ile 1	yer değişir	1, 3, 2, 5, 4, 7
3 ile 2	yer değişir	1, 2, 3, 5, 4, 7
3 ile 5	yerinde	1, 2, 3, 5, 4, 7
5 ile 4	yer değişir	1, 2, 3, 4, 5, 7

Bu örnekte dizi ikinci turun sonunda sıralanmıştır. Klasik bubble sort yine de kalan turları yapar ve sonuçta 1, 2, 3, 4, 5, 7 elde edilir.

8.2 Algoritma

Akış diyagramında önce dizinin boyutu ve dizi dışarıdan okunur. Dizinin ilk gözü, akış diyagramında 1'den ya da C'deki gibi 0'dan başlatılabilir; bu tasarıma bağlıdır. Burada 0'dan başlatılıyor ve dış döngü $n-2$'ye kadar gidiyor. 1'den başlatılsaydı dış döngü $n-1$'e kadar gidecekti. İç döngü her turda ilk elemandan başlar ve $a[j]$ ile $a[j+1]$'i karşılaştırır. Küçükten büyüğe sıralarken sonraki eleman küçükse, yani $a[j] > a[j+1]$ ise ikisinin yeri değiştirilir.

İç döngü $n-2-i$'ye kadar gider. Her tur bir elemanı en sona yerleştirdiği için sona kadar gitmeye gerek yoktur. Örneğin 7 yerleştikten sonra en sona bakılmaz.

Aşağıdaki program derste anlatılan algoritmanın C karşılığıdır. `gcc -Wall` ile derlenip çalıştırıldı, çıktısı `1 2 3 4 5 7`.

```c
#include <stdio.h>

void bubble_sort(int *a, int n) {
    int i, j, tmp;
    for (i = 0; i <= n - 2; i++)
        for (j = 0; j <= n - 2 - i; j++)
            if (a[j] > a[j + 1]) {
                tmp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = tmp;
            }
}

int main(void) {
    int a[] = {3, 5, 1, 2, 7, 4};
    int i, n = 6;
    bubble_sort(a, n);
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

8.3 Maliyet: bellek erişimi

Bubble sort'ta yapılan iş çoğunlukla atamadır, aritmetik işlem yoktur. Ama işlemci belleğe bir veri yolu (bus) üzerinden erişir ve bu erişimin hızı CPU döngüsünden çok daha yavaştır. Bu yüzden bir işlemin maliyetli olması için çok sayıda CPU döngüsü gerektirmesi şart değildir. Her atama, her dizi gözüne erişim, bilgi önbellekte (cache) değilse ve bellek erişimi gerektiriyorsa bir kayıptır. Bubble sort çok fazla bellek erişimi yaptığı için bu, algoritmanın en önemli dezavantajıdır.

> **Ek bilgi (derste anlatılmadı):** Önbellek (cache), işlemciye yakın, küçük ve hızlı bir bellektir. İstenen veri önbellekteyse ana belleğe gitmeye gerek kalmaz. Veri önbellekte yoksa (cache miss) ana bellekten getirilir ve bu, işlemci açısından çok sayıda döngü beklemek demektir.

İç içe iki döngü olduğu için bubble sort $O(n^2)$ karmaşıklığa sahiptir. Hoca bunu dış döngüdeki yaklaşık $n-2$ ile iç döngüdeki $n-2-i$ adımın çarpımı olarak anlattı. Karmaşıklık bir sonraki hafta daha yakından incelenecek.

> **Ders sonrası not:** İki döngü sayısının doğrudan çarpılması yalnız kaba bir üst sınır verir. İç döngü her turda bir kısaldığı için toplam karşılaştırma sayısı $\sum_{i=0}^{n-2}(n-1-i) = \frac{n(n-1)}{2}$ olur. Bu yine $O(n^2)$'dir. Derste kullanılan $n = 6$ elemanlı dizide klasik bubble sort 15 karşılaştırma ve 6 yer değiştirme yapar (python ile sayıldı).

9. Exchange Bayraklı Bubble Sort

9.1 Sıralı dizide gereksiz karşılaştırmalar

Bubble sort'un yer değiştirme yaklaşımından bir avantaj çıkarabiliriz. Diyelim ki bize kötü durum değil de iyi durum verildi: 1, 2, 3, 4, 5, 7. İlk turda 2, 1'den küçük mü? Değil. 3, 2'den küçük mü? Değil. 4, 3'ten; 5, 4'ten; 7, 5'ten küçük mü? Hiçbiri değil. Birinci tur bittiğinde hiçbir elemanın yeri değişmemiştir. İkinci turda yine aynı cevaplar alınır.

Hiç eleman değişmeyen bir tur yakaladıysanız dizi artık sıralanmıştır. Ama klasik bubble sort for döngüleri yüzünden hiçbir şekilde erken durmaz. Dizi sıralı verilse bile tüm karşılaştırmaları yapar.

9.2 Algoritma

Çözüm, dış döngüyü bir `while` döngüsüne çevirip turda değişiklik olup olmadığını izlemektir. Bunun için `exchange` adında bir değişken tanımlanır ve başlangıçta 1 yapılır. i sıfırdan başlatılır. Dış döngü, $i < n - 1$ (yani en son $n - 2$ numaralı turu işleyecek kadar) **ve** `exchange == 1` olduğu sürece döner.

> **Hocanın vurgusu:** En çok yapılan hatalardan biri AND ile OR'u karıştırmaktır. Burada her iki şart da sağlandığı sürece devam etmemiz gerektiği için AND kullanılır.

Dış döngüye girince `exchange` 0 yapılır: bu turda hiç eleman değişmezse 0 kalacaktır. İç döngüde komşu elemanlar yine tek tek karşılaştırılır. Yer değiştirme gerekiyorsa yer değiştirilir ve `exchange` 1 yapılır. Tek bir değişiklik bile olsa dizi hâlâ sıralanmamış demektir, yapılacak iş vardır. İç döngü bitince i bir artırılır ve dış döngüye dönülür. `exchange` 1 olduğu için döngüye yeniden girilir. Hiç eleman değişmeyen bir turda, yani `if`'in true koluna hiç girilmeyen bir turda, `exchange` 0 kalır, `while`'dan çıkılır ve dizi ekrana yazdırılır.

Aşağıdaki program `gcc -Wall` ile derlenip çalıştırıldı.

```c
#include <stdio.h>

int bubble_sort_exchange(int *a, int n) {
    int i = 0, j, tmp, exchange = 1;
    while (i < n - 1 && exchange == 1) {
        exchange = 0;
        for (j = 0; j <= n - 2 - i; j++)
            if (a[j] > a[j + 1]) {
                tmp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = tmp;
                exchange = 1;
            }
        i++;
    }
    return i; /* yapilan tur sayisi */
}

int main(void) {
    int a[] = {1, 3, 2, 4, 5, 7};
    int i, n = 6, tur;
    tur = bubble_sort_exchange(a, n);
    printf("tur=%d: ", tur);
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Çıktı `tur=2: 1 2 3 4 5 7` olur.

9.3 Hangi dizilerde kazanç sağlar?

Hoca bu sürümün kazanç sağladığı dizilere örnek olarak doğrudan sıralı verilmiş diziyi, 1, 2, 3, 7, 5, 4 dizisini ve 1, 3, 2, 4, 5, 7 dizisini verdi. Bu dizilerde iki sürümün yaptığı iş python ile sayıldı ve C programıyla tur sayıları doğrulandı (ara adım eklendi):

Dizi	Klasik: tur / karşılaştırma	Bayraklı: tur / karşılaştırma
1, 2, 3, 4, 5, 7	5 / 15	1 / 5
1, 3, 2, 4, 5, 7	5 / 15	2 / 9
1, 2, 3, 7, 5, 4	5 / 15	3 / 12
3, 5, 1, 2, 7, 4	5 / 15	3 / 12

Sıralı dizide bayraklı sürüm tek turda biter. 1, 3, 2, 4, 5, 7'de ilk tur 3 ile 2'yi değiştirir, ikinci turda hiç değişiklik olmaz ve algoritma durur. Önemli olan mümkün olan en kısa sürede durmaktır. Klasik sürüm iyi durumda bile for döngüleri yüzünden sona kadar gitmek zorundadır. Bu yüzden bayraklı sürüm, en kötü durum dışında her zaman daha avantajlıdır.

10. for ile while Arasındaki Seçim

Hoca bu örnekten yola çıkarak en sık yapılan hatalardan birine dikkat çekti:

- Bir `for` döngüsünü `break` ile bitirmeye çalışmak. `break` kullanacaksanız `while` kullanın.
- `for` döngüsünü döngü değişkenini değiştirerek bitirmeye çalışmak. Örneğin döngü n'de bitecekken i'ye $n + 1$ atayıp döngünün bitmesini sağlamak.

Bir kontrolün sonucuna göre, önceden belli olmayan bir adımda döngüden çıkılacaksa `for` değil, `while` kullanılmalıdır. Öyle olmasaydı `while` döngüsüne hiç gerek kalmazdı; her şey `for` ile yapılır, ya `break` konur ya da indis sınır değerinin dışına itilirdi. Doğru döngüyü seçmek okunabilirlik ve takip edilebilirlik açısından çok önemlidir.

> **Hocanın vurgusu:** `for` ile `break` kullanmak, `while` kullanmakla aynı şey değildir. Bundan uzak durun. Bu, temel tasarımda dikkat edilmesi gereken çok önemli bir konudur.

11. Selection Sort

11.1 Mantık

Selection sort'ta mantık biraz değişir. Bubble sort'ta komşu değerlere ve onların büyüklüğüne bakılıyordu. Selection sort'ta ise her turda kalan elemanlar dolaşılır ve o turun en küçük elemanı bulunur. Daha küçük bir eleman görüldükçe elimizdeki elemanı bırakıp o küçük elemanı alırız.

11.2 Örnek: 3, 5, 1, 2, 7, 4

Birinci tur: 3'ü elimize alırız. 5, 3'ten küçük mü? Değil. 1, 3'ten küçük mü? Evet; 3'ü bırakıp 1'i alırız. 2, 1'den küçük mü? Değil. 7 ve 4 de 1'den küçük değil. Turun en küçüğü 1'dir. 1 ile başta elimize aldığımız 3 yer değiştirir: dizi 1, 5, 3, 2, 7, 4 olur.

İkinci tur: 5 ile başlarız, en küçüğün 5 olduğunu varsayarız. 3 daha küçük, 3'ü alırız. 2, 3'ten küçük, 2'yi alırız. Kalan elemanlar daha küçük değil. Bu turun en küçüğü, yani dizinin ikinci en küçüğü 2'dir. 2 ile 5 yer değiştirir: 1, 2, 3, 5, 7, 4.

Bu, sondan bir önceki elemana kadar sürer ve sonuçta 1, 2, 3, 4, 5, 7 elde edilir. Kalan turlar aşağıdadır (ara adım eklendi, python ile doğrulandı):

Tur	Elde tutulan adaylar	Tur sonu dizi
1	3, 1	1, 5, 3, 2, 7, 4
2	5, 3, 2	1, 2, 3, 5, 7, 4
3	3	1, 2, 3, 5, 7, 4
4	5, 4	1, 2, 3, 4, 7, 5
5	7, 5	1, 2, 3, 4, 5, 7

Üçüncü turda 3 zaten yerindedir; eleman kendisiyle yer değiştirir.

11.3 Algoritma

Akış diyagramında yine dizi ve boyutu dışarıdan okunur. Elemanlar 0'dan $n-1$'e kadardır. Dış döngü $n-2$ numaralı elemana kadar gider, çünkü her eleman kendisinden sonrakilerle karşılaştırılacaktır. Elimize aldığımız ilk elemanın indisi `min` değişkeninde saklanır. İç döngü $i+1$'den başlar, çünkü eleman geri kalanlarla karşılaştırılacaktır. $a[j] < a[min]$ ise, yani yeni bakılan eleman daha küçükse, `if`'in true kolunda `min = j` yapılır: artık en küçüğümüz j numaralı gözdeki elemandır. İç döngü bitince `min`, o turun en küçük elemanını gösterir ve yer değiştirme döngünün dışında yapılır.

Selection sort, her yeni küçük eleman bulunduğunda yer değiştirecek biçimde de yazılabilirdi. Ama bu verimli olmaz. Onun yerine yalnız bulunan küçük elemanın indisi saklanır. İç döngüden çıkınca başta alınan elemanla bulunan en küçük eleman, ek bir değişken kullanılarak bir swap işlemiyle yer değiştirir.

> **Hocanın vurgusu:** En çok hata yapılan yerlerden biri swap işlemidir. Swap'ı mutlaka geçici bir değişkenle ve dikkatle yapın.

Aşağıdaki program `gcc -Wall` ile derlenip çalıştırıldı, çıktısı `1 2 3 4 5 7`.

```c
#include <stdio.h>

void selection_sort(int *a, int n) {
    int i, j, min, tmp;
    for (i = 0; i <= n - 2; i++) {
        min = i;
        for (j = i + 1; j <= n - 1; j++)
            if (a[j] < a[min])
                min = j;
        tmp = a[i];
        a[i] = a[min];
        a[min] = tmp;
    }
}

int main(void) {
    int a[] = {3, 5, 1, 2, 7, 4};
    int i, n = 6;
    selection_sort(a, n);
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

11.4 Avantaj ve dezavantaj

Burada da iç içe iki döngü vardır. En kötü durumda zaman karmaşıklığı $O(n^2)$'dir.

Algoritmanın avantajı daha az swap, yani daha az bellek erişimi yapmasıdır. Bu, bubble sort'a göre önemli bir avantajdır. Ama en iyi durumda da $O(n^2)$ karşılaştırma yapılır. Dizi sıralı verilse bile selection sort tüm karşılaştırmaları yapar. Sıralıya yakın bir dizide bayraklı bubble sort çok daha az işle sonuca ulaşır.

Derste kullanılan dizilerde iki algoritmanın yaptığı iş python ile sayıldı (ara adım eklendi): 3, 5, 1, 2, 7, 4 dizisinde selection sort 15 karşılaştırma ve 5 swap (biri elemanın kendisiyle), klasik bubble sort 15 karşılaştırma ve 6 swap yapar. Sıralı 1, 2, 3, 4, 5, 7 dizisinde selection sort yine 15 karşılaştırma yaparken bayraklı bubble sort 5 karşılaştırmayla durur. Selection sort'ta swap sayısı her zaman $n-1$'dir; bubble sort'ta ise en kötü durumda her karşılaştırma bir swap'a dönüşür.

12. Algoritma Analizinde Üç Senaryo

Hoca bu haftanın bir ısınma turu olduğunu söyledi. Bubble sort ve selection sort bilinen konulardır. Ama ileride görülecek hataları ve daha önce yanlış öğrenilmiş noktaları düzeltmek için üzerlerinden geçildi.

Bir algoritma iki temel bakış açısıyla analiz edilir:

- **Zaman karmaşıklığı (time complexity):** Algoritmanın bir işi yaklaşık kaç adımda çözdüğü.
- **Yer karmaşıklığı (space complexity):** Algoritmanın o işi yapmak için ne kadar alan kullandığı. Bu, girdi olarak kullandığımız değişken cinsinden ifade edilir. Ek dizi, ek matris ya da birden fazla dizi kullanmak yer karmaşıklığını olumsuz etkiler. Ama çalışılan veri grubuna ve sistemin bellek kapasitesine bağlı olarak bu konuda esneklik payı vardır.

Hem zaman hem de yer karmaşıklığında üç temel senaryo vardır:

- **En iyi durum (best case):** Gerçekleşme olasılığı çok düşük olduğu için genellikle belirleyici değildir, pek dikkate alınmaz.
- **Ortalama durum (average case):** En iyi durumdan daha önemlidir. Ama hesaplaması kolay olmadığı ve algoritmalar en kötü duruma göre hazırlandığı için bir kenarda bırakılır.
- **En kötü durum (worst case):** Her zaman birinci öncelik verilen senaryodur. Soru şudur: veri grubu en kötü hâliyle verilirse algoritma işini kaç adımda bitirir?

İki algoritmanın en kötü durumu karşılaştırıldığında biri avantajlıysa genellikle zaman karmaşıklığı düşük olan tercih edilir. Bu üstünlük analizle teorik ya da pratik olarak gösterilebilir. Bu üç senaryo dönem içinde daha ayrıntılı kullanılacak.

13. Doğrusal Arama

Arama da sıralama kadar önemlidir. Çünkü ekleme (insert), güncelleme (update) ya da silme yaparken her seferinde önce elemanı bulmak, elemanın veri grubunda olup olmadığına bakmak gerekir.

Doğrusal aramada (linear search) en önemli nokta, aranan veri grubunun sıralı mı sırasız mı olduğudur. Uygulanacak strateji buna göre değişir.

13.1 Sırasız dizide arama

n boyutlu sırasız bir dizide elemanlara tek tek bakılır: aranan eleman dizideki elemana eşit mi? Değilse bir sonrakine geçilir. En kötü durumda eleman sondadır ya da dizide hiç yoktur, bu durumda n karşılaştırma yapılır. Hoca örnek olarak 5, 7, 1, 2, 8, 9 dizisinde $x = 6$ aranmasını verdi: x 5'e, 7'ye, 1'e, 2'ye, 8'e, 9'a eşit mi? Hiçbirine eşit olmadığı için $n = 6$ karşılaştırma yapılır ve dizinin sonuna gelinir. Bu yaklaşımın en kötü durum zaman karmaşıklığı $O(n)$'dir.

13.2 Sıralı dizide arama

Dizi sıralıysa, örneğin 1, 2, 3, 5, 7, 8 ve aranan 6 ise, akıllıca olan şudur: 1'e, 2'ye, 3'e, 5'e bakılır, hiçbiri 6 değildir. 7'ye bakınca aranan elemandan büyük bir elemana ulaşılmıştır. Bundan sonra gelecek elemanlar (7, 8, 9, 10, 25, 38 gibi) dizi sıralı olduğu için 6'ya eşit olamaz. Dizide bir milyon eleman olsa bile arama burada bitirilmelidir. Sırasız dizi yaklaşımı uygulansaydı sona kadar gidilir, boşu boşuna bir milyon eleman karşılaştırılırdı.

> **Hocanın vurgusu:** Verilen veri grubunun sıralı mı sırasız mı olduğu, tasarlayacağınız algoritma için çok önemlidir. İlk sormanız gereken sorulardan biri budur.

Derste gösterilen akış diyagramı sıralı dizi içindir, çünkü hem büyüktür kontrolü hem de sınır kontrolü birlikte yapılır. Önce boyut, sonra dizi, sonra aranacak eleman okunur. $i = 0$'dan başlanır. $i < n - 1$ olduğu sürece **ve** aranan eleman dizideki elemandan büyük olduğu sürece aramaya devam edilir. Döngüden çıkınca $a[i] = x$ kontrolü yapılır.

Hocanın verdiği iz ($x = 6$):

- $6 > 1$ mi? Evet. İndis $n - 1$'den küçük mü? Evet. Devam.
- $6 > 2$, $6 > 3$, $6 > 5$ mi? Hepsi evet. Devam.

- $6 > 7$ mi? Hayır. Döngüden çıkılır, $i = 4$ ve $a[i] = 7$.
- $a[i]$ yani 7, 6'ya eşit mi? Hayır. Ekrana "not found" yazılır.

Aşağıdaki program `gcc -Wall` ile derlenip çalıştırıldı. Fonksiyon, bulunursa elemanın indisini, bulunmazsa -1 döndürür.

```c
#include <stdio.h>

int sirali_ara(const int *a, int n, int x) {
    int i = 0;
    while (i < n - 1 && x > a[i])
        i++;
    return a[i] == x ? i : -1;
}

int main(void) {
    int a[] = {1, 2, 3, 5, 7, 8};
    int n = 6;
    printf("6 -> %d\n", sirali_ara(a, n, 6));
    printf("8 -> %d\n", sirali_ara(a, n, 8));
    printf("9 -> %d\n", sirali_ara(a, n, 9));
    return 0;
}
```

Çıktı `6 -> -1`, `8 -> 5`, `9 -> -1` olur. Son eleman (8), döngü $i = n-1$'de durduktan sonra çıkıştaki $a[i] = x$ kontrolüyle bulunur. Dizideki tüm elemanlardan büyük bir değer (9) arandığında da döngü $i = n - 1$'de durur ve dizinin dışına çıkılmaz.

13.3 Sınır ihlali

Dizilerle çalışırken sınır ihlali yapmamak çok kritiktir. Diziniz 0'dan 5'e kadar indisliyken $a[6]$'ya bakıyorsanız size ait olmayan bir alana bakıyorsunuz demektir. Bu bazen sorun yaratmaz: belleğin o bölgesinde bir kısıt yoksa ya da orada çalışan başka bir program yoksa bir şey olmaz. Ama orayı kullanan bir yer varsa bellek erişim hatası olarak segmentation fault alırsınız. Bu yüzden ikili kontroller ve sınır kontrolü yaparken, döngünün içinde ya da döngüden çıktıktan sonra dizinin gözüne erişirken dizinin dışına taşıp taşmadığınızı ya da taşma ihtimalinizi mutlaka düşünün. Tek bir örnekte bile taşarsanız, o örnek denk geldiğinde segmentation fault olur ve program çökerek kullanıcıya hata verir.

> **Ek bilgi (derste anlatılmadı):** Yukarıdaki aramada döngü koşulu `i < n` diye yazılsaydı, x tüm elemanlardan büyük olduğunda döngü $i = n$'de durur ve çıkıştaki `a[i] == x` kontrolü $a[n]$'i okurdu. Bu, dizinin dışına erişen tipik bir sınır ihlalidir. C'de dizi dışına erişim tanımsız davranıştır (undefined behavior); program bazen çalışır, bazen çöker.

14. Insertion Sort'a Giriş

Insertion sort bu haftanın konusu değil. Önümüzdeki hafta buradan devam edilecek. Hoca yalnız mantığını verdi.

Kâğıt oyunu oynayanlar bilir: elinize karışık iskambil kâğıtları geldiğinde, sıralı oynamaya alışkın kişiler kâğıtları hemen düzenler. Kâğıtlar sıralı gelmiş ama biri yerinde değilse, onu alıp olması gereken yere yerleştirir. Insertion sort da aynı mekanizmayı kullanır: o an bakılan eleman önceki elemanlara göre doğru yerde değilse, olması gereken yere yerleştirilir.

Örnek: 5, 1, 4, 3, 2, 7, 6.

- Önce 1'e bakılır. 1'in yeri 5'in önüdür: 1, 5.
- 4, 1 ile 5'in arasına girmelidir. 5'ten bir önceye kadar gelinir ve 4 oraya yerleştirilir: 1, 4, 5.
- 3'ün doğru yeri, 1, 4, 5 üçlüsünde 1 ile 4'ün arasıdır: 1, 3, 4, 5.
- 2 gelir, 1 ile 3'ün arasına yerleşir: 1, 2, 3, 4, 5.

Her turda bir sonraki eleman, geride bıraktığı elemanlar arasında sıralı kalacak biçimde yerleşmesi gereken konuma taşınır. Derste kalan iki adım anlatılmadı. Tamamlanınca (ara adım eklendi, python ile doğrulandı): 7 zaten 5'ten büyük olduğu için yerinde kalır, 1, 2, 3, 4, 5, 7; 6 ise 5 ile 7'nin arasına girer ve dizi 1, 2, 3, 4, 5, 6, 7 olur.

Bu algoritma küçük boyutlu verilerde verimli olduğu için küçük veri gruplarında tercih edilir. Karmaşıklığı önümüzdeki hafta incelenecek.

Örnek Sorular

Bu derste ekranda soru, quiz ya da alıştırma gösterilmedi ve hazır kare yoktu. Aşağıdaki sorular derste sözlü sorulan sorulardır.

1. (derste sözlü, 00:42:37) Akış diyagramında `for` döngüsünün bitiş kısmına yalnız bitiş değeri mi yazılır, yoksa bir koşul da yazılabilir mi? Bitiş değeri $x \cdot y$ olan bir döngü nasıl çizilmelidir?

> **Çözüm:** Bitiş kısmına bir değişken ya da sabit sayı yazılır, koşul yazılmaz. Bitiş değeri $x \cdot y$ ise önce bir işlem kutusunda `n = x * y` hesaplanır, döngünün bitiş kısmına `n` yazılır. "$x \leq y$ olduğu sürece" gibi bir koşula bağlı tekrar gerekiyorsa `for` değil, `while` döngüsü kullanılır.

2. (derste sözlü, 01:02:10) Akış diyagramında `if` kontrolünün true ve false kollarının hangi yönde çizildiği fark eder mi?

> **Çözüm:** Fark etmez. Genelde true sağa, false sola yazılabilir ama bunun bir önemi yoktur. Hangi kolun true, hangisinin false olduğu her durumda yazılmalıdır. Ayrıca true kolu boş, false kolu dolu bırakılmaz; gerekirse koşul ters çevrilir.

3. (derste sözlü, 01:28:13) Derste gösterilen arama akış diyagramı (döngü koşulu: $i < n-1$ ve $x > a[i]$; döngüden sonra $a[i] = x$ kontrolü) sırasız dizi için mi, sıralı dizi için mi tasarlanmıştır? 1, 2, 3, 5, 7, 8 dizisinde $x = 6$ arandığında ne olur?

> **Çözüm:** Bir öğrenci "sıralı dizi" diye cevap verdi, hoca doğruladı. Döngü koşulunda hem sınır kontrolü hem de $x > a[i]$ kontrolü vardır; aranan değerden büyük bir elemana gelindiğinde arama durur. Bu yalnız sıralı dizide doğrudur. $x = 6$ için $6 > 1$, $6 > 2$, $6 > 3$, $6 > 5$ doğrudur, $6 > 7$ yanlıştır. Döngü $i = 4$'te durur, $a[4] = 7 \neq 6$ olduğu için "not found" yazılır. Sonuç C programıyla doğrulandı.

Dersin Toparlaması

Bu derste algoritmanın tanımı ve iyi bir algoritmanın özellikleri hatırlatıldı: önce doğruluk, sonra anlaşılabilirlik, kolay gerçeklenme ve değiştirilebilirlik. Algoritmalar zaman ve yer verimliliğine göre karşılaştırılır; birinci ölçüt zaman, ikinci ölçüt bellektir. Analizde en kötü durum esas alınır. Akış diyagramı kuralları, özellikle tek başlangıç ve tek bitiş, `if`'in true kolunun dolu olması, adım sayısı belliyse `for`, değilse `while` kullanılması ve `for`'u `break` ile bitirmemek üzerinde duruldu.

Bubble sort, exchange bayraklı bubble sort ve selection sort karşılaştırıldı. Hepsi en kötü durumda $O(n^2)$'dir. Bayraklı bubble sort sıralıya yakın dizilerde erken durarak kazanç sağlar, selection sort ise çok daha az swap yapar ama her durumda $O(n^2)$ karşılaştırma yapar. Doğrusal aramada sıralı dizide aranan değerden büyük bir elemana gelince durulabileceği ve sınır ihlalinin segmentation fault'a yol açabileceği gösterildi. Bir sonraki derste insertion sort'un ayrıntısıyla ve algoritma analizinin temel notasyonlarıyla devam edilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.